

k vecinos más cercanos

Unidad 5. Clasificación por similitud, alcances y limitaciones

Alejandro Veloz

INF396 - Introducción a la Ciencia de Datos



Objetivos

- Definir el clasificador y el regresor de k vecinos más cercanos como estimaciones locales de $\Pr(y \mid \mathbf{x})$ y de $\mathbb{E}[y \mid \mathbf{x}]$.
- Elegir k por validación cruzada y relacionar k con el compromiso sesgo–varianza.
- Enunciar las garantías teóricas del método y sus condiciones.
- Explicar el efecto de la escala de las variables y de la distancia elegida.
- Explicar la maldición de la dimensión y su efecto sobre los vecinos.
- Aplicar kNN a la segmentación de tumores en resonancia magnética, donde funciona bien.
- Mostrar dónde falla kNN con dígitos desplazados, y dos formas de corregirlo.

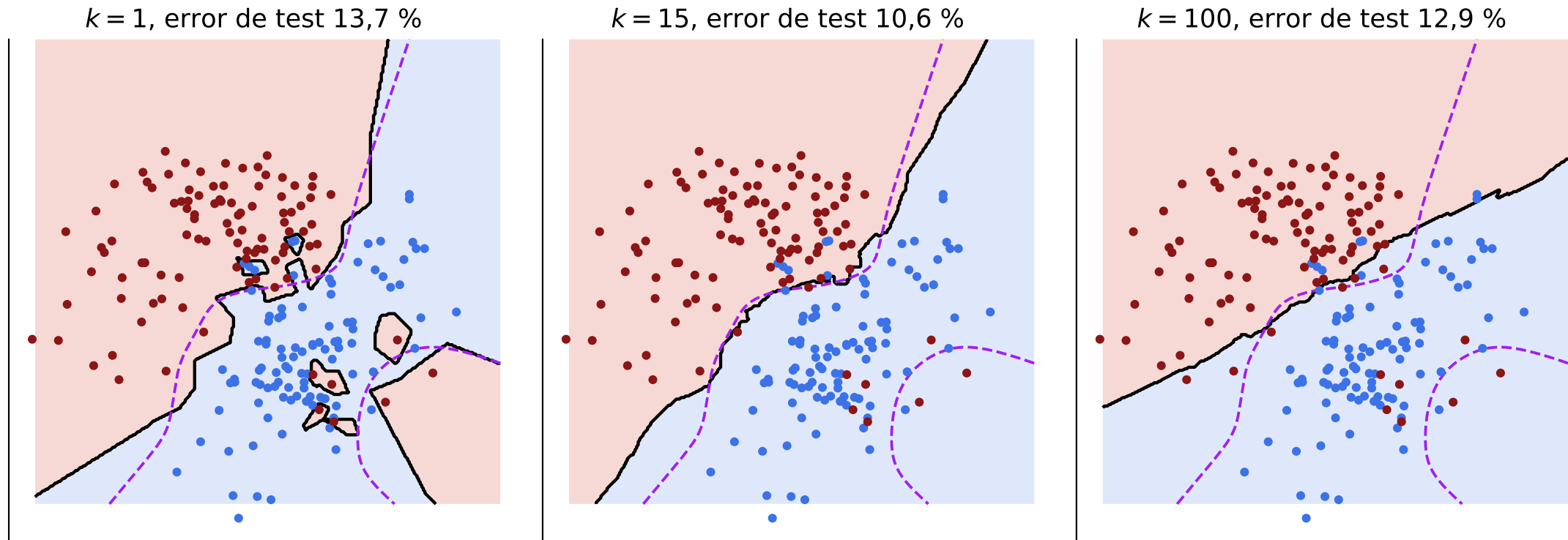
El método

Para predecir en $\mathbf{x}$, kNN busca los k puntos de entrenamiento más cercanos, $N_k(\mathbf{x})$, y promedia sus respuestas.

- **Regresión:** $\hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i$, un estimador local de $\mathbb{E}[y \mid \mathbf{x}]$.
- **Clasificación:** $\hat{p}_c(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} 1\{y_i = c\}$, y $\hat{y} = \operatorname{argmax}_c \hat{p}_c(\mathbf{x})$, el voto de la mayoría.

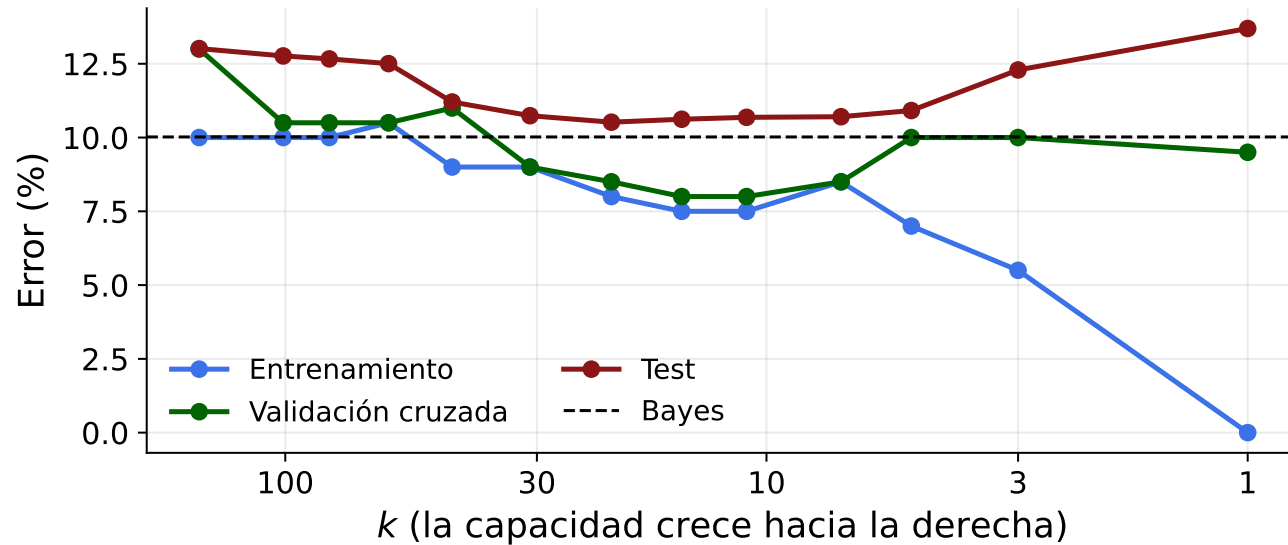
kNN no tiene ajuste. El modelo es el conjunto de entrenamiento completo. Todo el costo está en la predicción, que calcula distancias a los n puntos.

Tres valores de k



Hay doscientos puntos de dos clases, mezclas de diez normales. La frontera de Bayes (discontinua) tiene error 10,0 %. Con $k = 1$, la frontera rodea cada punto y el error de test es 13,7 %. Con $k = 100$, la frontera es casi recta, con error 12,9 %. Con $k = 15$, el error es 10,6 %.

Elegir k



En regresión, el error esperado de kNN es $\mathbb{E}[(y - \hat{f}(x))^2] = \sigma^2 + (f(x) - \frac{1}{k} \sum_{\ell} f(x_{(\ell)}))^2 + \sigma^2/k$.

Garantías teóricas

Cover y Hart (1967). Con $n \rightarrow \infty$, el error del clasificador de un vecino, R_1 , cumple

$$R^* \leq R_1 \leq 2R^*(1 - R^*),$$

donde R^* es el error de Bayes. El clasificador de un vecino tiene a lo sumo el doble del error óptimo.

Consistencia (Stone, 1977). Si $k \rightarrow \infty$ y $k/n \rightarrow 0$, el error de kNN converge al error de Bayes para cualquier distribución.

Las garantías son asintóticas. No dicen con qué velocidad se cumplen. La velocidad depende de la dimensión. En dimensión alta, los datos necesarios crecen exponencialmente. En la práctica, hay que revisar si la distancia elegida da vecinos realmente parecidos.

La distancia y la escala

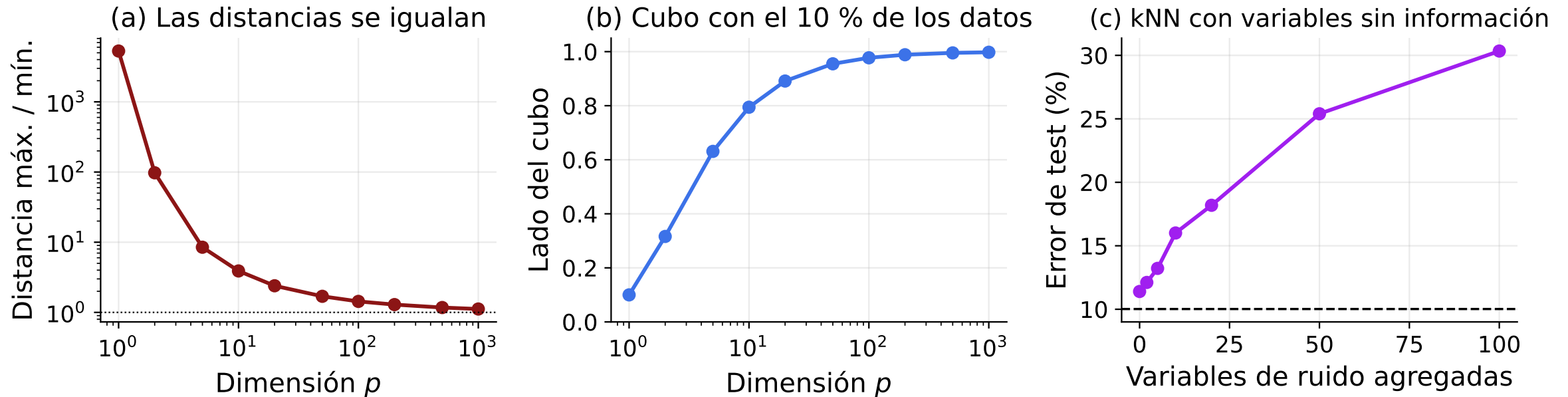
La distancia euclídea suma diferencias en todas las variables con el mismo peso. Una variable medida en una unidad grande, o con mucha varianza, domina la distancia.

El ejemplo usa $k = 15$. Dos variables determinan la clase. Una tercera, sin información, tiene desviación estándar 30.

Variables	Error de test
Las tres, en sus unidades originales	31,6 %
Las tres, estandarizadas	11,8 %
Sólo las dos informativas	11,4 %

Hay que estandarizar. Aun así, cada variable sin información agrega ruido a la distancia. kNN no selecciona variables. Se eligen antes, con conocimiento del problema o con un método de selección de variables.

La maldición de la dimensión



En el panel (a) hay 1 000 puntos uniformes en el cubo. El cociente entre la distancia al punto más lejano y al más cercano baja de 3,9 en $p = 10$ a 1,4 en $p = 100$. Todos los puntos quedan casi a la misma distancia. En el panel (b), con $p = 10$, el cubo con el 10 % de los datos tiene lado 0,79. Ese lado cubre casi todo el rango de cada variable, y los vecinos dejan de ser locales. En el panel (c) se agregan variables de ruido al problema de dos dimensiones. Con 100 variables, el error de kNN sube de 11,4 % a 30,3 %.

Costo computacional

- **Entrenamiento:** guardar los datos, sin costo de ajuste.
- **Predicción:** con fuerza bruta, $O(np)$ por consulta, y memoria para los $n \times p$ datos.

Estructuras para buscar vecinos más rápido:

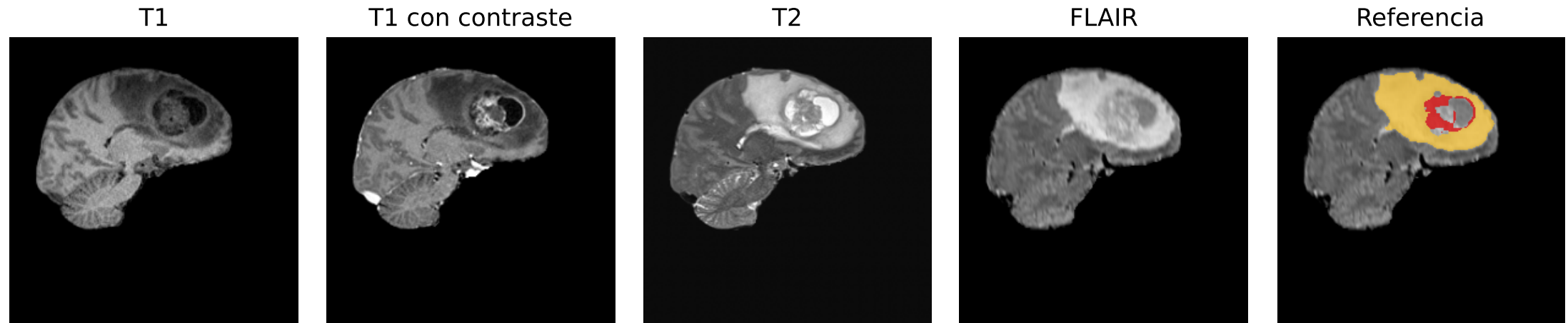
- **Árboles kd y árboles de bolas:** particionan el espacio y descartan zonas lejanas. Son muy eficientes en dimensión baja, aproximadamente $p \leq 20$. En dimensión alta pierden la ventaja.
- **Vecinos aproximados** (grafos HNSW, cuantización como en FAISS): aceptan un pequeño error en la búsqueda y ganan órdenes de magnitud en velocidad. Son la base de los buscadores actuales de imágenes y de documentos.

En la segmentación de tumores, un árbol kd sobre 60 000 puntos en cuatro dimensiones clasifica 4,7 millones de vóxeles en 46 segundos en un computador personal.

Aplicación con buen desempeño

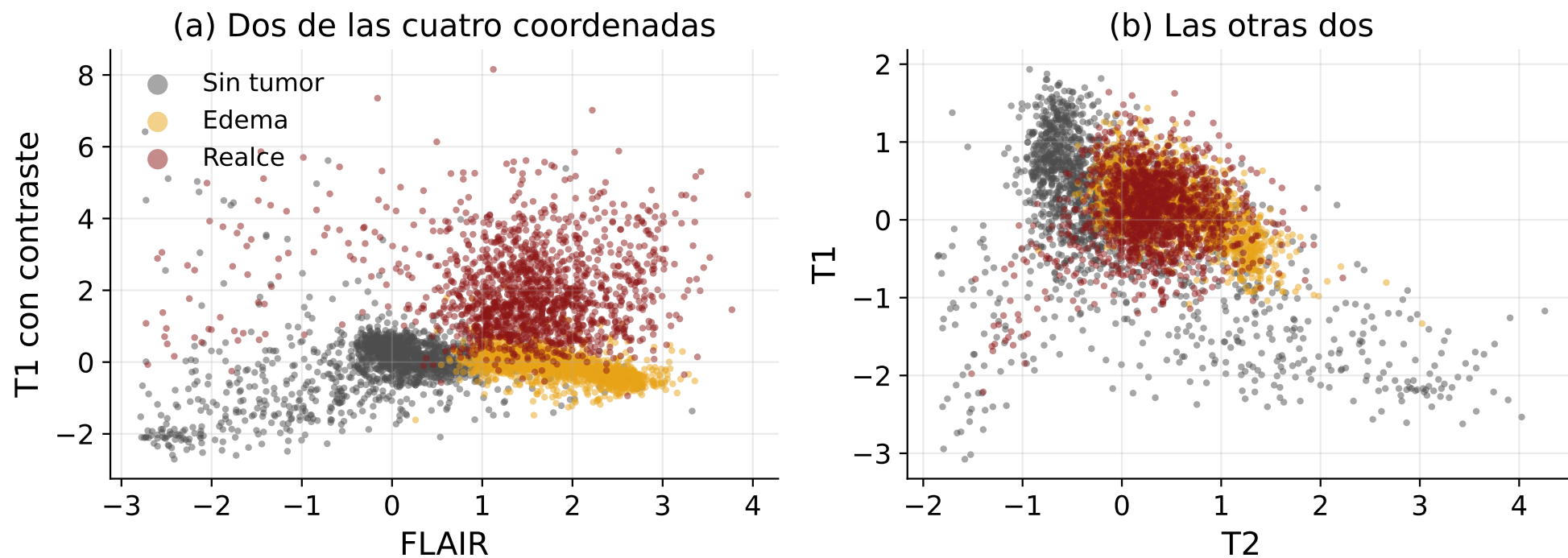
Segmentar glioblastoma en resonancia magnética

LUMIERE (Suter et al., 2022) tiene pacientes con glioblastoma, cada uno con cuatro secuencias registradas entre sí y sin cráneo. La segmentación de referencia es automática (HD-GLIO-AUTO). Sus tres clases son tejido sin tumor, edema más tumor que no realza, y tumor que realza con el contraste.



Cada vóxel es un punto en $\mathbb{R}^4$, dado por sus cuatro intensidades estandarizadas dentro del cerebro del paciente. La pregunta es si sus vecinos de otros pacientes predicen su clase.

El espacio de características



Hay 1 500 vóxeles por clase de un paciente de entrenamiento. El edema es brillante en FLAIR y no realza. El tumor que realza es brillante en T1 con contraste. Las clases forman nubes compactas con traslapes. Es la situación ideal para un método local, con pocas dimensiones con significado físico y muchos ejemplos.

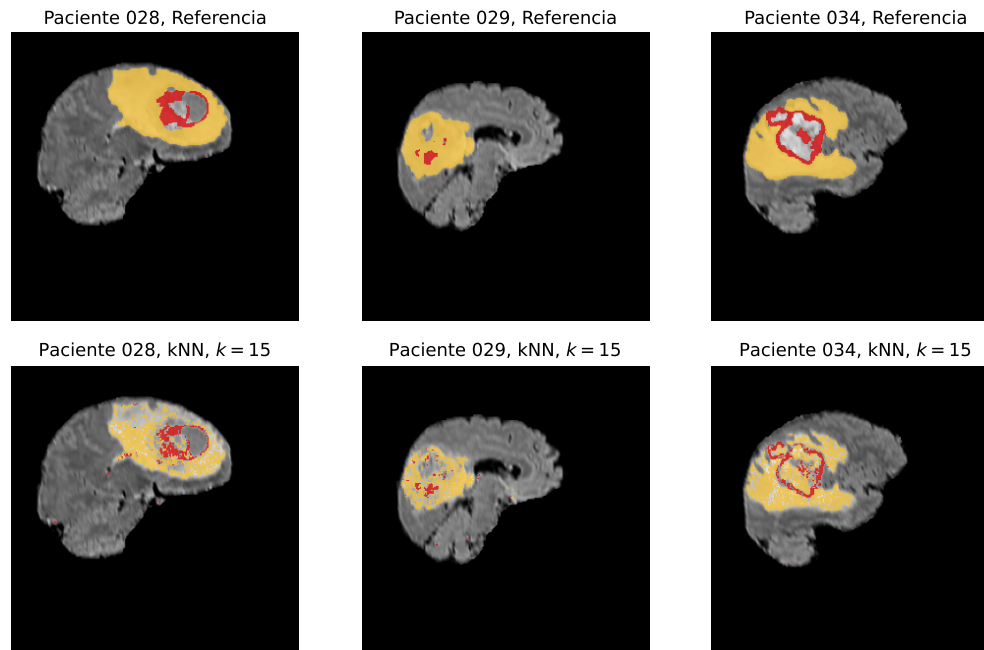
Protocolo

- **Muestra de entrenamiento:** 6 000 vóxeles sin tumor, 3 000 de edema y 3 000 de realce por paciente, 60 000 en total.
- **Elección de k :** validación dejando fuera un paciente de los cinco de entrenamiento. El Dice medio del tumor completo sube de 0,42 con $k = 1$ a 0,47 con $k = 15$ y se mantiene con k mayor; se elige $k = 15$.
- **Evaluación:** todos los vóxeles del cerebro de los tres pacientes de test, con el coeficiente de Dice $2|A \cap B|/(|A| + |B|)$.

La muestra balanceada tiene 50/25/25 %. El cerebro tiene 95,5/3,3/1,2 %. Las proporciones de kNN reflejan las clases de la muestra. Se corrigen con las probabilidades a priori del cerebro:

$$\tilde{p}_c(\mathbf{x}) \propto \hat{p}_c(\mathbf{x}) \frac{\pi_c^{\text{cerebro}}}{\pi_c^{\text{muestra}}}.$$

Resultados



Paciente	Vóxeles	Muestra balanceada		Corrección a priori	
		Tumor	Realce	Tumor	Realce
028	1 729 576	0,84	0,48	0,70	0,75
029	1 314 522	0,77	0,30	0,75	0,63
034	1 615 635	0,81	0,68	0,70	0,68

Interpretación de los resultados

- kNN usa sólo cuatro intensidades por vóxel, sin información espacial, y alcanza un Dice de 0,77 a 0,84 para el tumor completo frente a la referencia.
- La corrección a priori sube el Dice del tumor que realza (de 0,30–0,68 a 0,63–0,75), porque deja de confundir edema con realce.
- La corrección a priori baja el Dice del tumor completo, porque asigna a tejido sano algunos vóxeles de edema.

Por qué funciona. La dimensión es baja. La distancia tiene sentido tras estandarizar. Hay muchos ejemplos de cada tejido. Las clases forman nubes locales en el espacio de intensidades.

Límites. kNN clasifica cada vóxel sin mirar sus vecinos espaciales. Por eso aparecen vóxeles aislados mal clasificados. La estandarización por paciente no corrige del todo las diferencias entre equipos. Un paciente de otro hospital puede tener intensidades fuera de la nube de entrenamiento.

Aplicación con desempeño limitado

Dígitos manuscritos

MNIST (LeCun et al., 1998) tiene imágenes de 28×28 píxeles, o puntos en $\mathbb{R}^{784}$. Con 20 000 dígitos de entrenamiento, $k = 3$ y distancia euclídea entre píxeles, kNN acierta el 96,0 % de 2 000 dígitos de test.

kNN funciona a pesar de la dimensión porque los dígitos de MNIST están centrados y escalados. Dos imágenes del mismo dígito tienen los trazos en píxeles parecidos. La distancia entre píxeles mide parecido de forma.

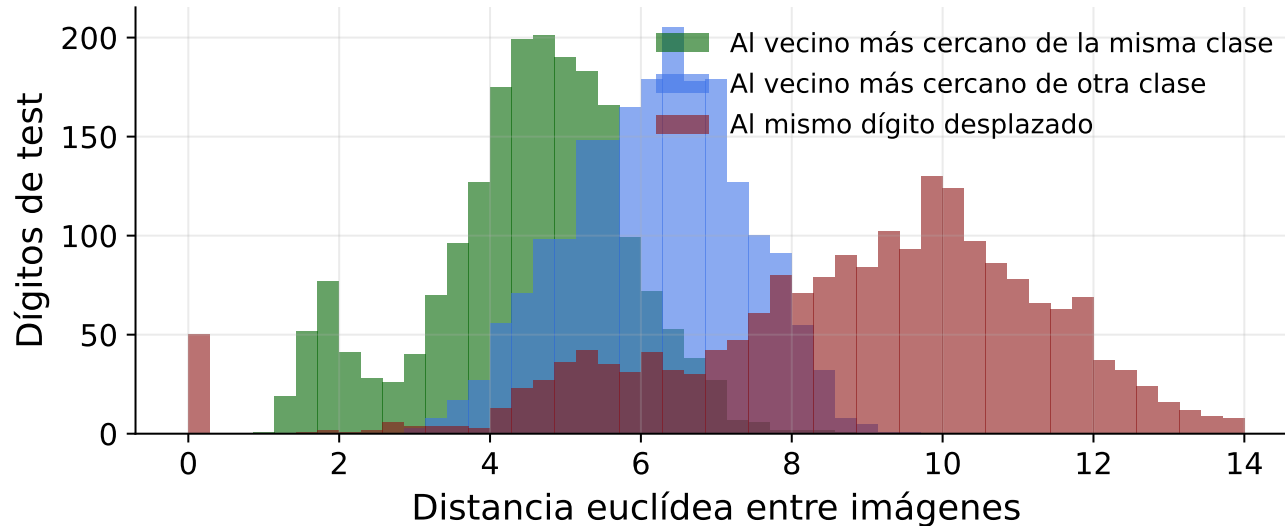
Se desplazan los dígitos de test hasta tres píxeles en cada eje, sin cambiar el entrenamiento. La exactitud cae a 52,6 %.

Los vecinos de un dígito desplazado



Hay cuatro dígitos de test, centrados (izquierda) y desplazados (derecha), y sus tres vecinos más cercanos. Los vecinos rojos son de otra clase. Los vecinos del dígito desplazado tienen trazos en el mismo lugar y otra forma. Un uno vertical ocupa el sitio de un cuatro. Un siete ocupa la curva de un tres.

Distancias entre dígitos desplazados



La distancia mediana de un dígito de test a su vecino más cercano es 4,6 en la misma clase y 6,3 en otra clase.

La distancia entre un dígito y su propia versión desplazada tiene mediana 9,3. Para el 87 % de los dígitos, la versión desplazada está más lejos que el vecino más cercano de otra clase.

La distancia euclídea entre píxeles no es invariante a los desplazamientos. kNN depende por completo de su distancia.

Dos soluciones

Aumentar la memoria. Se agregan al entrenamiento copias desplazadas de cada dígito. Así, un dígito desplazado encuentra vecinos de su clase. La memoria y el tiempo de búsqueda crecen en proporción al número de copias. Además, hay que conocer de antemano las transformaciones.

Cambiar la representación. Se buscan los vecinos en otro espacio, donde la distancia mide parecido de forma. Aquí se usan las 512 activaciones finales de una ResNet18 entrenada en ImageNet. Esta red convolucional nunca vio un dígito. Sus capas de convolución y de promedio espacial son aproximadamente invariantes a los desplazamientos.

Representación	Dimensión	Memoria	Test centrado	Test desplazado
Píxeles	784	20 000	96,0 %	52,6 %
Píxeles, memoria aumentada	784	60 000	95,7 %	91,5 %
ResNet18 preentrenada	512	20 000	93,2 %	90,5 %

Comparación de las dos aplicaciones

Las dos aplicaciones usan el mismo algoritmo. La diferencia está en la distancia.

- En los vóxeles, la distancia usa cuatro intensidades con significado físico. Dos vóxeles cercanos son del mismo tejido.
- En los dígitos, dos imágenes cercanas tienen tinta en los mismos píxeles. Eso equivale a tener la misma forma sólo si las imágenes están alineadas.

La calidad de kNN depende de su distancia. Con una representación adecuada, kNN es un clasificador excelente. Los buscadores actuales de imágenes y de documentos usan kNN sobre representaciones aprendidas por redes neuronales.

Cuándo usar kNN

Conviene	No conviene
Pocas variables, todas informativas	Muchas variables, muchas sin información
Una distancia con significado	Distancias entre datos crudos con invarianzas
Muchos datos y fronteras irregulares	Pocos datos en dimensión alta
Representaciones aprendidas (embeddings)	Necesidad de predicción muy rápida sin índice
Base de comparación sin supuestos	Necesidad de interpretar el efecto de cada variable

kNN no impone una frontera lineal ni cuadrática, y por eso no tiene sesgo de forma. A cambio, tiene mucha varianza y depende por completo de la distancia. La regresión logística y el discriminante tienen el compromiso opuesto.

Implementación

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV

def knn(X, y, Q, k):                                # fuerza bruta
    d = ((Q[:, None, :] - X[None, :, :])**2).sum(-1) # distancias al cuadrado (m x n)
    vecinos = np.argsort(d, axis=1)[:k]
    return np.array([np.bincount(f).argmax() for f in y[vecinos]])

modelo = make_pipeline(StandardScaler(),              # estandarizar dentro de la validación
                        KNeighborsClassifier(algorithm="kd_tree"))
busqueda = GridSearchCV(modelo, {"kneighborsclassifier__n_neighbors": [1, 3, 5, 11, 21, 51]},
                           cv=10).fit(X, y)
```

Errores frecuentes

- No estandarizar las variables. Así, la variable de mayor escala decide la distancia.
- Estandarizar con todos los datos antes de la validación cruzada. Esto filtra información del pliegue de validación.
- Incluir muchas variables sin información y esperar que kNN las ignore.
- Elegir k mirando el conjunto de test, o usar $k = 1$ porque tiene error de entrenamiento cero.
- Entrenar con una muestra balanceada y reportar las proporciones como probabilidades de la población sin corregir por las probabilidades a priori.
- Usar kNN sobre píxeles u otras representaciones crudas cuando el problema tiene invarianzas, como desplazamientos o rotaciones.
- Separar entrenamiento y test por vóxel o por imagen cuando los datos vienen de pacientes. Los vecinos de un vóxel están en el mismo paciente, y el error estimado resulta optimista.

Referencias

- Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning*, 2.^a ed., secciones 2.3, 2.5 y 13.3. Springer.
- Cover, T., Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- Stone, C. J. (1977). Consistent nonparametric regression. *Annals of Statistics*, 5(4), 595–620.
- Beyer, K., Goldstein, J., Ramakrishnan, R., Shaft, U. (1999). When is “nearest neighbor” meaningful? *ICDT*, 217–235.
- Simard, P., LeCun, Y., Denker, J. (1993). Efficient pattern recognition using a new transformation distance. *NeurIPS*.
- LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Suter, Y., et al. (2022). The LUMIERE dataset: longitudinal glioblastoma MRI with expert RANO evaluation. *Scientific Data*, 9, 768.
- Kickingeder, P., Isensee, F., et al. (2019). Automated quantitative tumour response assessment of MRI in neuro-oncology with artificial neural networks. *The Lancet Oncology*, 20(5), 728–740.